

AWS Application Load Balancer Project with Auto Scaling

Project Overview

This project demonstrates a production-style AWS web hosting architecture using AWS native services. The infrastructure was designed with high availability, scalability, security, and fault tolerance in mind. Unlike the previous project where NGINX was used as a custom load balancer, this architecture uses an AWS-managed Application Load Balancer (ALB) integrated with Auto Scaling Group (ASG), Target Groups, Launch Templates, and ACM SSL certificates. The backend web servers were deployed inside private subnets while public traffic was handled securely through the ALB.

Architecture Components

The infrastructure was designed across multiple Availability Zones in the ap-south-1 region to ensure high availability. The architecture contains:

- Custom VPC

- Public Subnets
- Private Subnets
- Internet Gateway (IGW)
- NAT Gateway
- Bastion Host
- Application Load Balancer (ALB)
- Target Group
- Auto Scaling Group (ASG)
- Launch Template
- Amazon Machine Image (AMI)
- AWS Certificate Manager (ACM)
- Route53 DNS

Network Design

A custom VPC was created manually with both public and private subnets. **Public Subnets**

Public subnets were used for:

- Application Load Balancer

- Bastion Host
 - NAT Gateway
- These subnets were connected to the Internet Gateway to allow internet access.

Private Subnets

Private subnets were used for backend EC2 instances created by Auto Scaling Group. The backend servers were intentionally isolated from the internet to improve security. Only the Application Load Balancer and Bastion Host were allowed to communicate with backend servers.

Step-by-Step Project Implementation

1. Create Template EC2 Instance

A template EC2 instance was created manually to configure the web application before creating an AMI. Apache HTTPD, PHP-FPM, and Git packages were installed.

2. Deploy Website

The website source code was cloned from GitHub and copied to the Apache web root directory. Proper ownership and permissions were configured for the Apache user.

3. Configure SSL

A self-signed SSL certificate was configured on the template instance to verify HTTPS functionality. Later, SSL termination was handled at the Application Load Balancer using ACM certificates.

4. Verify Website

The application was tested using the EC2 public DNS name to ensure both HTTP and HTTPS were functioning correctly.

5. Create AMI

After verifying the application, an Amazon Machine Image (AMI) was created from the configured EC2 instance. This AMI became the golden image for future backend instances.

6. Create Launch Template

A Launch Template was created using the AMI. The Launch Template included: • AMI ID • Instance Type • Security Groups • SSH Key Pair • Storage Configuration

7. Create Auto Scaling Group

Auto Scaling Group was configured using the Launch Template. Three private subnets across multiple Availability Zones were selected for backend instance deployment.

8. Create Target Group

A Target Group was created to register backend EC2 instances. Health checks were enabled to ensure traffic is routed only to healthy backend servers.

9. Create Application Load Balancer

An internet-facing Application Load Balancer was created in public subnets. The ALB distributes incoming traffic to backend EC2 instances registered in the Target Group.

10. Configure ACM SSL Certificate

AWS Certificate Manager (ACM) was used to attach SSL certificates to the ALB. HTTPS listener was configured on port 443.

11. Configure Route53

The domain shopping.vpkdevops.online was mapped to the Application Load Balancer using Route53 Alias records.

Backend Apache HTTPD Setup

```
sudo yum install httpd php-fpm git -y

sudo systemctl restart httpd.service php-fpm.service
sudo systemctl enable httpd.service php-fpm.service
```

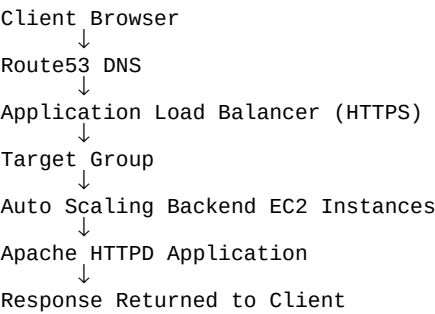
```
sudo git clone https://github.com/Fujikomalan/aws-elb-site

sudo cp -R aws-elb-site/* /var/www/html/

sudo chown -R apache:apache /var/www/html/*

sudo systemctl restart httpd.service
```

Application Request Flow



Security Group Design

Security Group	Ports	Allowed Source
ALB Security Group	80 / 443	0.0.0.0/0
Backend Security Group	80 / 443	ALB Security Group
Backend Security Group	22	Bastion Security Group
Bastion Security Group	22	My Public IP

High Availability and Scalability

The infrastructure was distributed across multiple Availability Zones to ensure maximum uptime. Auto Scaling Group automatically launches or terminates backend instances depending on application load. Benefits include:

- Fault tolerance

- Automatic recovery
- Dynamic scaling
- Reduced downtime
- Better performance during traffic spikes

AWS Services Used

AWS Service	Purpose
VPC	Custom networking environment
EC2	Backend web servers
Application Load Balancer	Traffic distribution
Target Group	Backend instance registration
Auto Scaling Group	Automatic scaling
Launch Template	Reusable EC2 configuration
AMI	Golden machine image
ACM	SSL certificate management

Route53	DNS management
NAT Gateway	Internet access for private servers
Internet Gateway	Public internet access
Security Groups	Firewall access control

Skills Demonstrated

AWS Skills

- VPC Design
- ALB Configuration
- Auto Scaling
- ACM SSL Integration
- Route53 Configuration
- EC2 Management

Linux Administration

- Apache HTTPD
- SSL Configuration
- Service Management
- Linux Networking

DevOps Skills

- Infrastructure Design
- Highly Available Architecture
- Scalable Web Hosting
- Secure Deployment Design

Conclusion

This project successfully implemented a highly available, scalable, and secure AWS-native web application architecture using Application Load Balancer and Auto Scaling Group. The backend application servers were securely deployed inside private subnets while the Application Load Balancer handled external HTTPS traffic using ACM SSL certificates. The architecture follows AWS best practices for security, scalability, and fault tolerance, making it suitable for production-level deployments.